

200. 岛屿数量

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium	(Not Available)	(Not Available)	-	-	0

- **Tags:** 深度优先搜索, 广度优先搜索, 并查集, 数组, 矩阵
- **Companies:** (Not Available)

给你一个由 '1'（陆地）和 '0'（水）组成的二维网格，请你计算网格中岛屿的数量。

岛屿总是被水包围，并且每座岛屿只能由水平方向和/或竖直方向上相邻的陆地连接形成。

此外，你可以假设该网格的四条边均被水包围。

示例 1:

```
1  输入: grid = [  
2    ["1","1","1","1","0"],  
3    ["1","1","0","1","0"],  
4    ["1","1","0","0","0"],  
5    ["0","0","0","0","0"]  
6  ]  
7  输出: 1
```

示例 2:

```
1  输入: grid = [  
2    ["1","1","0","0","0"],  
3    ["1","1","0","0","0"],  
4    ["0","0","1","0","0"],  
5    ["0","0","0","1","1"]  
6  ]  
7  输出: 3
```

提示:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 300`
- `grid[i][j]` 的值为 '0' 或 '1'

[Discussion](#) | [Solution](#)

解决方法

1. 问题理解

目标： 计算二维网格中由 '1' 组成的岛屿数量。

岛屿定义： 由水平或垂直相邻的 '1' 连接而成，四周被 '0' 包围。

输入： `m x n` 的字符网格 `grid`。

输出： 岛屿的数量。

2. 思路分析

这个问题是典型的图遍历问题，可以将网格看作一个图，每个 '1' 是一个节点，相邻的 '1' 之间有边。目标是计算图中连通分量的数量。

方法一：深度优先搜索 (DFS)

遍历整个网格。当遇到一个 '1' 时，说明发现了一个新岛屿（或者一个已知岛屿的一部分）。此时，岛屿数量加 1，并从这个 '1' 开始进行深度优先搜索，将所有与之相连的 '1'（即属于同一个岛屿的所有陆地）标记为已访问（例如，可以将其值改为 '0' 或使用一个额外的 `visited` 数组），以避免重复计数。

1. 初始化：

- `count = 0` (岛屿数量)
- 获取网格的行数 `rows = len(grid)` 和列数 `cols = len(grid[0])`。
- (可选) 创建一个与 `grid` 等大的 `visited` 布尔数组，初始化为 `False`。或者直接修改 `grid` 将 '1' 变为 '0'。

2. 遍历网格：

- 使用嵌套循环遍历 `grid` 中的每个单元格 `(r, c)`。

3. 发现陆地：

- 如果 `grid[r][c] == '1'` (并且 `visited[r][c]` 为 `False`，如果使用 `visited` 数组)，则：
 - `count += 1`。
 - 调用 `dfs(r, c)` 开始淹没（标记）这个岛屿。

4. DFS 函数 `dfs(r, c)`：

- **功能：** 从 `(r, c)` 开始，标记所有相连的陆地。
- **边界检查和终止条件：**
 - 如果 `r` 或 `c` 越界 (`r < 0` 或 `r >= rows` 或 `c < 0` 或 `c >= cols`)。
 - 如果当前单元格是水 (`grid[r][c] == '0'`)。
 - 如果当前单元格已被访问 (`visited[r][c]` 为 `True` 或 `grid[r][c]` 已被改为 '0')。
 - 满足以上任一条件，则返回。
- **标记当前单元格：**
 - 将 `grid[r][c]` 设置为 '0' (或 `visited[r][c] = True`)。
- **递归访问相邻单元格：**
 - `dfs(r + 1, c)` (下)

- `dfs(r - 1, c)` (上)
- `dfs(r, c + 1)` (右)
- `dfs(r, c - 1)` (左)

5. **返回结果**：遍历结束后，返回 `count`。

方法二：广度优先搜索 (BFS)

思路与 DFS 类似，只是将淹没岛屿的过程从递归改为迭代和队列。

1. **初始化**：同 DFS。

2. **遍历网格**：同 DFS。

3. **发现陆地**：

- 如果 `grid[r][c] == '1'`，则：
 - `count += 1`。
 - 调用 `bfs(r, c)` 开始淹没这个岛屿。

4. **BFS 函数** `bfs(r, c)`：

- **功能**：从 `(r, c)` 开始，标记所有相连的陆地。
- **初始化队列**：创建一个队列 `queue`，并将起始坐标 `(r, c)` 入队。
- **标记起始点**：将 `grid[r][c]` 设置为 '0'。
- **循环处理队列**：
 - 当 `queue` 不为空时：
 - 将队首元素 `(curr_r, curr_c)` 出队。
 - **检查并处理四个相邻单元格** `(next_r, next_c)`：
 - 对于每个相邻单元格，进行边界检查和陆地检查 (`grid[next_r][next_c] == '1'`)。
 - 如果有效且是陆地：
 - 将其标记为 '0'。
 - 将其坐标 `(next_r, next_c)` 入队。

5. **返回结果**：遍历结束后，返回 `count`。

方法三：并查集 (Union-Find)

将每个 '1' 视为一个独立的集合。遍历网格，当遇到一个 '1' 时，检查其上方和左方的相邻单元格。如果相邻单元格也是 '1'，则将当前 '1' 所在的集合与相邻 '1' 所在的集合合并。同时维护一个岛屿计数器，初始值为所有 '1' 的数量，每次成功合并就减 1。

1. **初始化**：

- 创建并查集数据结构（通常用数组 `parent` 表示，`parent[i]` 存储节点 `i` 的父节点）。大小为 `rows * cols`。
- 初始化每个陆地单元格 `(r, c)` 为一个独立的集合，`parent[r * cols + c] = r * cols + c`。
- `count = 0` (初始岛屿数量，等于陆地数量)。

2. **遍历网格**：

- 遍历 `grid` 中的每个单元格 `(r, c)`。
- 如果 `grid[r][c] == '1'`:
 - `count += 1` (先假设每个 '1' 都是一个岛屿)。
 - **检查上方相邻单元格 `(r-1, c)`:**
 - 如果 `r > 0` 且 `grid[r-1][c] == '1'`:
 - 调用 `union(r * cols + c, (r - 1) * cols + c)` 合并两个集合。如果合并成功 (即它们原本不属于同一个集合), 则 `count -= 1`。
 - **检查左方相邻单元格 `(r, c-1)`:**
 - 如果 `c > 0` 且 `grid[r][c-1] == '1'`:
 - 调用 `union(r * cols + c, r * cols + (c - 1))` 合并两个集合。如果合并成功, 则 `count -= 1`。

3. 并查集操作:

- `find(i)`: 查找节点 `i` 所在集合的根节点 (带路径压缩优化)。
- `union(i, j)`: 合并节点 `i` 和 `j` 所在的集合。查找 `i` 和 `j` 的根节点 `root_i` 和 `root_j`。如果 `root_i != root_j`, 则将一个根指向另一个 (例如 `parent[root_i] = root_j`), 并返回 `True` 表示合并成功。否则返回 `False`。

4. 返回结果: 遍历结束后, 返回 `count`。

优缺点:

- DFS/BFS: 直观, 空间复杂度取决于递归深度/队列大小 (最坏 $O(M*N)$)。
- 并查集: 对于某些动态连接问题更优, 但此题中实现稍复杂, 空间复杂度 $O(M*N)$ 。

3. 代码实现 (Python)

```

1  from typing import List
2  import collections
3
4  class Solution:
5      # 方法一: 深度优先搜索 (DFS)
6      def numIslandsDFS(self, grid: List[List[str]]) -> int:
7          if not grid or not grid[0]:
8              return 0
9
10         rows, cols = len(grid), len(grid[0])
11         count = 0
12
13         def dfs(r, c):
14             # 边界检查和终止条件
15             if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] ==
16             '0':
17                 return
18
19             # 标记当前单元格为已访问 (淹没)
20             grid[r][c] = '0'
21
22             # 递归访问相邻单元格
23             dfs(r + 1, c)

```

```

23         dfs(r - 1, c)
24         dfs(r, c + 1)
25         dfs(r, c - 1)
26
27     # 遍历网格
28     for r in range(rows):
29         for c in range(cols):
30             if grid[r][c] == '1':
31                 count += 1
32                 dfs(r, c)
33
34     return count
35
36 # 方法二: 广度优先搜索 (BFS)
37 def numIslandsBFS(self, grid: List[List[str]]) -> int:
38     if not grid or not grid[0]:
39         return 0
40
41     rows, cols = len(grid), len(grid[0])
42     count = 0
43     queue = collections.deque()
44
45     for r in range(rows):
46         for c in range(cols):
47             if grid[r][c] == '1':
48                 count += 1
49                 grid[r][c] = '0' # 标记为访问
50                 queue.append((r, c))
51
52         while queue:
53             curr_r, curr_c = queue.popleft()
54             # 检查四个方向
55             for dr, dc in [(0, 1), (0, -1), (1, 0), (-1, 0)]:
56                 next_r, next_c = curr_r + dr, curr_c + dc
57                 # 检查有效性
58                 if 0 <= next_r < rows and 0 <= next_c < cols
and grid[next_r][next_c] == '1':
59                     grid[next_r][next_c] = '0'
60                     queue.append((next_r, next_c))
61
62     return count
63
64 # 方法三: 并查集 (Union-Find)
65 def numIslandsUnionFind(self, grid: List[List[str]]) -> int:
66     if not grid or not grid[0]:
67         return 0
68
69     rows, cols = len(grid), len(grid[0])
70     parent = list(range(rows * cols))
71     count = 0
72
73     def find(i):
74         if parent[i] == i:
75             return i
76         parent[i] = find(parent[i]) # 路径压缩
77         return parent[i]

```

```

78     def union(i, j):
79         nonlocal count
80         root_i = find(i)
81         root_j = find(j)
82         if root_i != root_j:
83             parent[root_i] = root_j
84             count -= 1 # 合并成功, 岛屿总数减一
85             return True
86         return False
87
88     for r in range(rows):
89         for c in range(cols):
90             if grid[r][c] == '1':
91                 count += 1 # 先假设每个 '1' 都是一个岛屿
92                 idx = r * cols + c
93                 # 检查上方
94                 if r > 0 and grid[r - 1][c] == '1':
95                     union(idx, (r - 1) * cols + c)
96                 # 检查左方
97                 if c > 0 and grid[r][c - 1] == '1':
98                     union(idx, r * cols + (c - 1))
99
100     # 统计 '1' 的个数, 然后减去合并次数也可以, 但上面的方法更直接
101     # initial_ones = sum(row.count('1') for row in grid)
102     # count = initial_ones # 初始岛屿数
103     # ... union 操作中 count -= 1 ...
104
105     # 另一种统计方法: 最后统计根节点的数量
106     # root_count = 0
107     # for r in range(rows):
108     #     for c in range(cols):
109     #         if grid[r][c] == '1':
110     #             idx = r * cols + c
111     #             if parent[idx] == idx: # 如果一个陆地单元格是其所在集合的根
112     #                 root_count += 1
113     # return root_count
114
115     return count
116
117
118     # 最终提交选择 DFS 或 BFS
119     def numIslands(self, grid: List[List[str]]) -> int:
120         # 复制一份 grid, 避免修改原输入 (如果需要)
121         # grid_copy = [row[:] for row in grid]
122         # return self.numIslandsDFS(grid_copy)
123
124         # 直接修改原 grid 的 DFS 实现
125         return self.numIslandsDFS(grid)
126         # return self.numIslandsBFS(grid)
127         # return self.numIslandsUnionFind(grid)
128
129

```

4. 复杂度分析

- DFS / BFS:

- 时间复杂度: $O(M * N)$, 其中 M 是行数, N 是列数。每个单元格最多被访问常数次。
- 空间复杂度:
 - DFS: $O(M * N)$ 在最坏情况下 (整个网格是一个岛屿), 递归深度可能达到 $M * N$ 。
 - BFS: $O(\min(M, N))$ 在最坏情况下, 队列中最多存储对角线长度的节点数, 或者 $O(M * N)$ 如果整个网格是一个岛屿。
 - 如果使用 `visited` 数组, 则需要额外的 $O(M * N)$ 空间。如果直接修改 `grid`, 则空间复杂度主要由递归栈或队列决定。

- 并查集:

- 时间复杂度: $O(M * N * \alpha(M * N))$, 其中 α 是 Ackermann 函数的反函数, 其增长非常缓慢, 对于实际应用中的 M 和 N , 可以认为 $\alpha(M * N)$ 是一个接近常数的值。遍历网格是 $O(M * N)$, 每次 `find` 和 `union` 操作的平均时间复杂度接近 $O(1)$ 。
- 空间复杂度: $O(M * N)$, 用于存储并查集的 `parent` 数组。

5. 如何在面试中讲解

1. 问题转换:

- "这个问题可以看作是在一个二维网格图中寻找连通分量的数量, 其中 '1' 代表节点, 相邻的 '1' 之间有边。"

2. 核心思路 (DFS/BFS):

- "我们可以遍历整个网格。当遇到一个未访问过的陆地 ('1') 时, 我们就找到了一个新的岛屿, 计数器加 1。"
- "然后, 我们需要将这个新发现的岛屿的所有部分都标记为已访问, 以避免重复计数。这可以通过图的遍历算法实现, 比如 DFS 或 BFS。"

3. DFS 实现细节:

- "定义一个 `dfs(r, c)` 函数。它的作用是将 `(r, c)` 及其所有相连的陆地标记为已访问 (比如改成 '0')。"
- "在 `dfs` 内部, 首先进行边界检查和状态检查 (是否越界、是否是水、是否已访问)。"
- "如果当前单元格有效且是陆地, 则将其标记, 并递归调用 `dfs` 访问其上、下、左、右四个相邻单元格。"
- "主函数中, 遍历网格, 遇到 '1' 就 `count++` 并调用 `dfs`。"

4. BFS 实现细节:

- "思路类似, 但使用队列进行迭代。"
- "遇到 '1' 时, `count++`, 将该点标记并入队。"
- "当队列不为空时, 出队一个节点, 检查其四个邻居。如果是未访问的陆地, 则标记、入队。"

5. 复杂度分析:

- "DFS 和 BFS 的时间复杂度都是 $O(M * N)$, 因为每个单元格最多访问常数次。"

- "空间复杂度: DFS 最坏 $O(M * N)$ (递归栈), BFS 最坏 $O(\min(M, N))$ 或 $O(M * N)$ (队列)。如果直接修改输入网格, 则这是主要的空间开销。"

6. (可选) 并查集思路:

- "也可以用并查集。将每个 '1' 初始化为一个集合。遍历网格, 遇到 '1' 时, 检查其上方和左方的邻居。如果是 '1', 则将当前格子与邻居格子所在的集合合并 (union)。"
- "维护一个计数器, 初始为 '1' 的总数, 每次成功合并就减 1。最终计数器就是岛屿数量。"
- "时间复杂度接近 $O(M * N)$, 空间复杂度 $O(M * N)$ 。"

7. 选择与权衡:

- "DFS 和 BFS 在此问题上效率相当, 实现也比较直观。DFS 代码可能更简洁, 但有递归深度风险; BFS 没有递归风险。"
- "并查集是另一种思路, 但在简单连通性问题上可能不如 DFS/BFS 简洁。"